THREE SIGMA

InfiniFi Protocol

Security Review

Disclaimer

Security Review

InfiniFi Protocol

Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

InfiniFi Protocol

Table of Contents

Summary

Security Review

InfiniFi Protocol

Summary

Three Sigma audited InfiniFi in a 0.8 person week engagement. The audit was conducted from 08/06/2026 to 09/06/2026.

Protocol Description

InfiniFi is a DeFi protocol that enables users to mint and redeem receipt tokens (iUSD, iETH) against collateral assets (USDC, ETH). The protocol features a sophisticated yield generation system through multiple farm integrations, a locking mechanism for enhanced rewards, and a governance system for farm allocation voting.

Scope

Security Review

InfiniFi Protocol

Scope

Filepath	nSLOC
InfiniFi-Labs/infinifi-contracts/pull/280	53
InfiniFi-Labs/infinifi-contracts/pull/380	336
InfiniFi-Labs/infinifi-contracts/pull/381	91
Total	**480**

Methodology

Security Review

InfiniFi Protocol

To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

InfiniFi Protocol

Project Dashboard

Application Summary

Name	InfiniFi
Repository	https://github.com/InfiniFi-Labs/infinifi-contracts
Commit	PR280, PR380, PR381
Language	Solidity
Platform	Ethereum

Engagement Summary

Timeline	08/06/2026 to 09/06/2026
№ of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	2	2	0
Medium	1	1	0
Low	0	0	0
None	6	3	3

Category Breakdown

Suggestion	6
Documentation	0
Bug	3
Optimization	0
Good Code Practices	0

Risk Section

Security Review

InfiniFi Protocol

Risk Section

- **Integrators Warning - Withdrawals Revert Under Queued Redemptions**: Because SIUSDC's _withdraw passes the full owed assets as the minAssetsOut slippage bound to the asynchronous, queue-based RedeemController, every ERC4626 withdrawal reverts whenever the redemption queue is non-empty or available liquidity is below the requested amount, causing a denial of service.

Findings

Security Review

InfiniFi Protocol

Findings

3S-H01

Idle USDC in **SIUSDC** bricks every withdrawal because **SIUSDC::_withdraw** passes the full owed amount as **minAssetsOut**

Id	3S--H01
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #141f26a.

Description

SIUSDC::_withdraw services a redemption by unstaking the caller's pro-rata share of **siUSD** into **iUSD** and then calling **gateway.redeem** to turn that **iUSD** into USDC. The vault holds assets in two places at once: idle USDC sitting directly in the contract, and value staked as **siUSD**. **SIUSDC::totalAssets** adds both, so **assets** (derived from **previewRedeem(shares)**) covers the idle USDC plus the staked value.

The redeem leg only produces USDC from the unstaked **siUSD**, yet **SIUSDC::_withdraw** passes the full **assets** as the **minAssetsOut** argument to **gateway.redeem**. The idle USDC the vault already holds is never credited toward that minimum, so the redeem output falls short by the caller's pro-rata claim on the idle balance. The gateway enforces **assetsOut >= minAssetsOut** and reverts with **MinAssetsOutError** whenever the vault holds any idle USDC.

An attacker triggers this by transferring a tiny amount of USDC directly into the vault. The donation lands as idle balance, the **minAssetsOut** requirement overshoots the redeem output by each holder's share of that balance, and every **withdraw** and **redeem** call reverts. The freeze lifts only when someone calls **deposit**, because **SIUSDC::_deposit** runs **mintAndStake** over the whole USDC balance and clears the idle amount. The donation costs a few wei plus gas, so the attacker re-donates right after each deposit and keeps the withdraw and redeem paths frozen indefinitely for the price of dust. This is a cheap, repeatable denial of service against every depositor's exit.

Steps to Reproduce

1. Alice deposits 1000 USDC into the vault and receives shares. The deposit stakes the USDC into **siUSD**, leaving zero idle USDC.

2. The attacker transfers 2 wei of USDC directly to the vault address.

3. Alice calls **redeem** for her full balance. **assets** from **previewRedeem** now includes her pro-rata claim on the 2 wei of idle USDC.

4. **SIUSDC::_withdraw** unstakes only the **siUSD** leg, so the redeem returns less than **assets**.

5. **gateway.redeem** reverts with **MinAssetsOutError** because **assetsOut** is below **minAssetsOut**. Every other depositor's withdraw and redeem reverts the same way.

6. Someone calls **deposit**, which clears the idle USDC. The attacker sends another 2 wei in the same block or right after, and the freeze returns at dust cost.

Proof of Concept

The test lives at **test/integration/external/SIUSDC.it.sol** in the **SIUSDCIntegrationTest** contract. It forks mainnet at block 24026034, deposits 1000 USDC as Alice, donates 2 wei of USDC into the vault, then asserts that **redeem** reverts with **MinAssetsOutError**. It also confirms the freeze clears once a fresh deposit converts the idle USDC into **siUSD**, which is the window the attacker re-donates into.

```
function testRedeemRevertsWhenVaultHoldsIdleUSDC() public {
    // Alice deposits 1000 USDC and receives shares
    uint256 depositAmount = 1000e6;
    dealToken(address(usdc), ALICE, depositAmount);
    vm.startPrank(ALICE);
    usdc.approve(address(vault), depositAmount);
    uint256 shares = vault.deposit(depositAmount, ALICE);
    vm.stopPrank();
    // Anyone transfers a tiny amount of USDC directly into the vault.
    // 2 wei is the minimum at this fork state.
    uint256 donation = 2;
    dealToken(address(usdc), address(this), donation);
    usdc.transfer(address(vault), donation);
    assertEq(usdc.balanceOf(address(vault)), donation, "vault holds idle USDC");
    // ERC4626 still reports the full position as redeemable...
    assertEq(vault.maxRedeem(ALICE), shares, "maxRedeem still reports full
balance");
    // ...but redeem reverts: minAssetsOut (= assets, including the idle-USDC share)
    // exceeds the USDC the siUSD leg can produce.
    vm.prank(ALICE);
```

```
    bool reverted;
    try vault.redeem(shares, ALICE, ALICE) returns (uint256) {
        reverted = false;
    } catch (bytes memory reason) {
        reverted = true;
        assertEq(bytes4(reason), InfiniFiGatewayV2.MinAssetsOutError.selector,
"should revert MinAssetsOutError");
    }
    assertTrue(reverted, "redeem should revert while vault holds idle USDC");
    // The DoS clears as soon as any deposit converts the idle USDC to siUSD.
    dealToken(address(usdc), ALICE, 1e6);
    vm.startPrank(ALICE);
    usdc.approve(address(vault), 1e6);
    vault.deposit(1e6, ALICE);
    assertEq(usdc.balanceOf(address(vault)), 0, "deposit converts idle USDC to
siUSD");
    uint256 out = vault.redeem(vault.balanceOf(ALICE), ALICE, ALICE);
    vm.stopPrank();
    assertGt(out, 0, "redeem succeeds once idle USDC is cleared");
}
```

Run it against a mainnet archive RPC (the **mainnet** alias in **foundry.toml** resolves the **MAINNET_RPC** environment variable):

```
MAINNET_RPC=<mainnet_archive_rpc_url> \
forge test --match-contract SIUSDCIntegrationTest \
  --match-test testRedeemRevertsWhenVaultHoldsIdleUSDC -vv
```

Expected output. The test passes, which proves the redeem reverts under idle USDC and recovers after a deposit:

```
Ran 1 test for test/integration/external/SIUSDC.it.sol:SIUSDCIntegrationTest
[PASS] testRedeemRevertsWhenVaultHoldsIdleUSDC() (gas: 5877081)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 631.88ms
```

Recommendation

Credit the idle USDC the vault already holds toward the redeem minimum, and skip the unstake and redeem when the idle balance already covers the owed amount.

```
   function _withdraw(address caller, address receiver, address owner, uint256
assets, uint256 shares)
       internal
       override
   {
+       uint256 idle = usdc.balanceOf(address(this));
+       if (assets > idle) {
          // Over-estimate the siUSD to redeem, which might result in some USDC dust
left in the vault
          // This might revert if the amount withdrawn is very small, and results in 0
iUSD / USDC
          uint256 siUSDAmount = shares.mulDivUp(siUSD.balanceOf(address(this)),
totalSupply());
          uint256 siUSDBalance = siUSD.balanceOf(address(this));
          siUSDAmount = siUSDAmount > siUSDBalance ? siUSDBalance :
siUSDAmount;
          uint256 iusdAmount = gateway.unstake(address(this), siUSDAmount);
-         gateway.redeem(address(this), iusdAmount, assets);
+          gateway.redeem(address(this), iusdAmount, assets - idle);
+      }
       // Burn shares, transfer USDC to receiver, emit event
       super._withdraw(caller, receiver, owner, assets, shares);
   }
```

3S-H02

Aave V4 bad debt is carried at par in **AaveV4Farm::assets()**, over-stating protocol NAV and blocking the last redemptions

Id	3S--H02
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #f385c87.

Description

AaveV4Farm is a maturity farm that swaps the primary **assetToken** (e.g. USDC) into **aaveAssetToken** (USDG) and supplies it to an Aave V4 Spoke. Its **AaveV4Farm::assets()** is the protocol-level accounting consumers' single source of truth for how much the Aave leg is worth. The problem is that the value it reports includes realized, uncollectible bad debt (**deficitRay**) carried at par, so it over-states the farm's true equity, and the gap is not transient.

assets() prices the supplied position by reading **getUserSuppliedAssets**, then converting it to **assetToken**:

```
function assets() public view override(MultiAssetFarmV2, IFarm) returns (uint256) {
    uint256 totalAssets = super.assets();
    IAaveV4Spoke.UserPosition memory userPosition =
IAaveV4Spoke(spoke).getUserPosition(reserveId, address(this));
    if (userPosition.suppliedShares == 0) return totalAssets;
    uint256 suppliedAaveAssets =
IAaveV4Spoke(spoke).getUserSuppliedAssets(reserveId, address(this));
    if (aaveAssetToken == assetToken) return totalAssets + suppliedAaveAssets;
    else return totalAssets + convert(aaveAssetToken, assetToken,
suppliedAaveAssets);
}
```

getUserSuppliedAssets resolves to a share to asset conversion priced against **totalAddedAssets**. In the canonical Aave V4 source that quantity is (**hub/libraries/AssetLogic.sol:90-96**, **:229-242**):

totalAddedAssets = liquidity + swept + aggregatedOwedRay - realizedFees - unrealizedFees;
aggregatedOwedRay = drawnShares * drawnIndex + premium + deficitRay; // hub/libraries/AssetLogic.sol:241

The supplier's per-share value is **totalAddedAssets / addedShares**, and it includes **deficitRay** (realized bad debt) at par. When a position is liquidated with its collateral exhausted but debt remaining, the deficit is recorded in two steps: **Hub::reportDeficit** (**hub/Hub.sol:304-330**) decrements the aggregate debt shares (**asset.drawnShares**/**spoke.drawnShares**, **hub/Hub.sol:315-318**) and records the loss in **deficitRay** (**asset.deficitRay**/**spoke.deficitRay**, **hub/Hub.sol:320-323**), while the defaulting borrower's own position is cleared separately in **LiquidationLogic::notifyReportDeficit** (**spoke/libraries/LiquidationLogic.sol:296**). In **aggregatedOwedRay** the decrement of **drawnShares * drawnIndex** and the increment of **deficitRay** cancel, so **totalAddedAssets** is unchanged at the moment of default. There is no asset and no counterparty behind it: the borrower is gone, and **deficitRay** only ever decreases via **eliminateDeficit** (**hub/Hub.sol:333-352**), which is **restricted** and works by burning another spoke's **addedShares** to donate fresh external capital. InfiniFi neither controls nor can force that, so from InfiniFi's perspective the impairment is indefinite — it persists until Aave governance or another spoke eliminates the deficit, which InfiniFi cannot compel. This is the TradFi equivalent of keeping a defaulted loan on the books at face value instead of writing it down.

AaveV4Farm::assets() reads this gross share value and therefore inherits InfiniFi's pro-rata share of **deficitRay** at par. The over-statement is:

overstatement ≈ (userAddedShares / totalAddedShares) * deficitRay // in aaveAssetToken units

This value is impaired but still counted in **assets()**. Note that the unwind path instead is consistent with realizable liquidity, not with **assets()**: **withdrawFromAave** → **spoke.withdraw** → **Hub::remove** is gated by **require(amount <= asset.liquidity)**.

The issue manifests itself in the consumer path. **Accounting::totalAssetsValue()** sums every farm's **assets()**:

```
function totalAssetsValue() external view returns (uint256 _totalValue) {
    address[] memory assets = FarmRegistry(farmRegistry).getEnabledAssets();
    for (uint256 i = 0; i < assets.length; i++) {
        uint256 assetPrice = IOracle(oracle[assets[i]]).price();
        uint256 _assets =
_calculateTotalAssets(FarmRegistry(farmRegistry).getAssetFarms(assets[i]));
```

```
            _totalValue += _assets.mulWadDown(assetPrice);
        }
    }
```

totalAssetsValue() (**Accounting.sol:48-55**) iterates every enabled asset's farms, so the over-stated **assets()** flows into the unfiltered total that **YieldSharing::unaccruedYield()** (**YieldSharing.sol:132-141**) reads. Because the deficit is carried at par, **assets()** does not drop when the bad debt is realized, so **unaccruedYield()** does not go negative by the impaired amount and **accrue()** (**YieldSharing.sol:147-157**) never routes that loss into **_handleNegativeYield()**. The loss is suppressed, not managed.

This is the core harm, because **_handleNegativeYield()** (**YieldSharing.sol:244-277**) is precisely the logic meant to absorb such a loss, and it does so through a deliberate seniority waterfall:

1. consume the protocol safety buffer first, sparing all users;

2. then slash the locking users — the illiquid, first-loss tranche that is paid boosted yield specifically to absorb losses first;

3. then slash the siUSD savers;

4. and only as a last resort mark down the receipt-token oracle price for all iUSD holders pro-rata.

By keeping the deficit at par this entire ordering is bypassed: the safety buffer is never consumed and the junior, first-loss tranches are never charged.

While NAV is overstated, **accrue()** keeps distributing yield (or failing to slash) as if the reserve were whole, and iUSD's redemption price never reflects the deficit, so every redeemer exits at par against a backing that is genuinely short. Redemptions pull only from **LIQUID** farms (**BeforeRedeemHook**) while the phantom value sits in this **MATURITY** farm, so as redeemers cash out at full value the real assets shrink while the phantom does not, concentrating the fixed shortfall onto the users least able to exit — the lockers and the holders who remain. First out is made whole; last out holds the hole.

Recommendation

Treat realized, uncollectible bad debt as impaired in valuation: **AaveV4Farm::assets()** must report the supplied position net of InfiniFi's pro-rata share of **deficitRay**, so the impaired amount never reaches the NAV/yield path.

1. Extend the local **IAaveV4Hub** interface as it currently declares only **getAssetLiquidity** (**src/interfaces/aave/IAaveV4Spoke.sol:4-6**). Add the two getters the live Hub already exposes:

```
 interface IAaveV4Hub {
     function getAssetLiquidity(uint256 assetId) external view returns (uint256);
+    function getAddedShares(uint256 assetId) external view returns (uint256);     //
hub/Hub.sol:517-520
+    function getAssetDeficitRay(uint256 assetId) external view returns (uint256); //
hub/Hub.sol:563-565
 }
```

2. Net the pro-rata deficit out of **AaveV4Farm::assets()**: subtract InfiniFi's pro-rata slice of the deficit from **suppliedAaveAssets**, in **aaveAssetToken** units, before the **convert(...)** branch. **getUserSuppliedAssets()** already gives the gross claim, so the corrected value is **gross - haircut**, where **haircut** is InfiniFi's pro-rata share of the bad debt:

```
 function assets() public view override(MultiAssetFarmV2, IFarm) returns (uint256) {
     uint256 totalAssets = super.assets();
     IAaveV4Spoke.UserPosition memory userPosition =
IAaveV4Spoke(spoke).getUserPosition(reserveId, address(this));
     if (userPosition.suppliedShares == 0) return totalAssets;
     uint256 suppliedAaveAssets =
IAaveV4Spoke(spoke).getUserSuppliedAssets(reserveId, address(this));
+    // write down our pro-rata share of realized bad debt (deficitRay)
+    IAaveV4Spoke.Reserve memory reserve =
IAaveV4Spoke(spoke).getReserve(reserveId);
+    IAaveV4Hub hub = IAaveV4Hub(reserve.hub);
+    uint256 deficitAssets = hub.getAssetDeficitRay(reserve.assetId).fromRayUp(); //
ceil(ray / 1e27)
+    if (deficitAssets != 0) {
+        uint256 haircut = Math.mulDiv(
+            deficitAssets,
+            uint256(userPosition.suppliedShares),
+            hub.getAddedShares(reserve.assetId) + 1e6, // Aave virtual shares
+            Math.Rounding.Ceil
+        );
+        suppliedAaveAssets = suppliedAaveAssets > haircut ? suppliedAaveAssets -
haircut : 0;
+    }
+
    if (aaveAssetToken == assetToken) return totalAssets + suppliedAaveAssets;
    else return totalAssets + convert(aaveAssetToken, assetToken,
suppliedAaveAssets);
 }
```

3S-M01

Synchronous withdrawal over an asynchronous redemption system blocks withdrawals

Id	3S--M01
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #b0f52e8.

Description

SIUSDC is an ERC4626 vault that wraps USDC into yield-bearing **siUSDC** shares by routing deposits through the InfiniFi gateway into siUSD. Withdrawals are expected to be synchronous and atomic, as the ERC4626 standard implies: a caller burns shares and receives the underlying asset within the same transaction. **SIUSDC::_withdraw** implements this by unstaking siUSD to iUSD via **gateway.unstake** and then calling **gateway.redeem(address(this), iusdAmount, assets)**, where the third argument is the **minAssetsOut** slippage bound and is set to **assets**, the full amount the user is owed for the shares being burned.

The redemption system it routes into (the **RedeemController** reached through the gateway) is not synchronous. **RedeemController::redeem** is queue-based: if the redemption queue is already non-empty (**queueLength() > 0**), it enqueues the entire request and returns **0**; if the queue is empty but available **liquidity()** is smaller than the requested amount, it pays out only the available portion, enqueues the remainder, and returns just **availableAssetAmount**. In both cases the realized **assetsOut** is strictly less than the requested **assets**. **InfiniFiGatewayV2::redeem** then enforces **require(assetsOut >= _minAssetsOut, MinAssetsOutError(...))**, which reverts because **SIUSDC** passed **assets** as **minAssetsOut**.

The consequence is that every withdrawal through **SIUSDC** reverts whenever the protocol-wide redemption queue is non-empty or whenever available liquidity is below the requested amount.

A related view-layer consequence is that **maxWithdraw** and **maxRedeem** continue to report nonzero values while the downstream contracts are in a queued or liquidity-constrained state, so integrators and routers that trust these views will submit transactions that revert.

Recommendation

The clean fix is to model the asynchronous redemption system honestly rather than wrapping it in a synchronous interface. Add a request/claim withdrawal flow: enqueue the redemption on the user's behalf in **SIUSDC::_withdraw** (accepting the partial fill and queued remainder that **RedeemController::redeem** returns) and expose a **claimRedemption** function so the user can later collect the queued portion once liquidity arrives. This matches the **RedeemController**'s real semantics, at the cost of breaking atomic ERC4626 withdrawals, so it should be documented clearly for integrators.

If a full async redesign is out of scope, at minimum override **maxWithdraw** and **maxRedeem** to return **0** when the redemption queue is non-empty or when available liquidity is insufficient. This does not remove the denial of service, but it stops integrators from attempting exits that are guaranteed to revert and makes the vault's **max*** views EIP-4626 compliant. Either way, the synchronous-only, best-effort nature of exits should be explicitly documented.

3S-N01

RWAEscrowRouter constructor redundantly reassigns **keeper** and **assetToken** already set by the parent constructor

Id	3S--N01
Classification	None
Category	Suggestion
Status	Acknowledged

Description

RWAEscrowRouter::constructor calls the parent **RWAEscrow::constructor** with **RWAEscrow(_core, _assetToken, address(this), _keeper)**. That parent constructor already assigns **keeper = _keeper** and **assetToken = _assetToken** from those same arguments. The child constructor then repeats both assignments with the identical values, so the two lines have no effect.

assetToken is declared **immutable** in **RWAEscrow**, and the parent constructor is its initialization point. The reassignment in the child is dead code that adds noise and invites a reader to assume the values differ from the parent's.

Recommendation

Remove the two redundant assignments and let the parent constructor set both values.

```
    constructor(address _core, address _assetToken, address _keeper)
        RWAEscrow(_core, _assetToken, address(this), _keeper)
    {
-       keeper = _keeper;
-       assetToken = _assetToken;
    }
```

3S-N02

Natspec on **RWAEscrowRateManager::governanceUpdateTotalAssets** wrongly claims the call works while the escrow is paused

Id	3S--N02
Classification	None
Category	Suggestion
Status	Acknowledged

Description

RWAEscrowRateManager::governanceUpdateTotalAssets lets the timelock overwrite an escrow's **totalAssets** directly, skipping the rate based calculation in **harvest**. Its natspec carries **@dev Can be called even if the contract is paused**, which reads as a promise that governance can update total assets during a pause.

The claim only holds for this contract's own pause state. Pausing works per contract through OpenZeppelin **Pausable**, and **governanceUpdateTotalAssets** forwards to **RWAEscrow::reportTotalAssets**, which carries the **whenNotPaused** modifier. When the target **RWAEscrow** is paused, **reportTotalAssets** reverts and the forwarded call reverts with it, even though the entry point on **RWAEscrowRateManager** is itself reachable. The documentation overstates the capability: the call bypasses the rate manager's pause, but it still requires the target escrow to be unpaused.

Recommendation

Correct the natspec so it states the actual behavior: the call bypasses the rate manager's pause but reverts when the target escrow is paused, since **RWAEscrow::reportTotalAssets** is gated by **whenNotPaused**.

```
// src/finance/RWAEscrowRateManager.sol
   /// @notice Allows our timelock to bypass the rate limitations (sudden loss, etc)
   ///         by calling the escrow directly and updating the total assets
-  /// @dev Can be called even if the contract is paused
+   /// @dev Bypasses this contract's pause, but the target escrow must be unpaused:
+   ///      RWAEscrow.reportTotalAssets reverts when the escrow is paused.
   function governanceUpdateTotalAssets(address _escrow, uint256 _assets)
       external
```

```
    onlyCoreRole(CoreRoles.PROTOCOL_PARAMETERS)
{
    RWAEscrow(_escrow).reportTotalAssets(_assets);
}
```

3S-N03

RWAEscrow::withdraw emits **TokensWithdrawn** with **receiver** instead of the address that receives the tokens

Id	3S--N03
Classification	None
Category	Suggestion
Status	Acknowledged

Description

RWAEscrow::withdraw transfers the asset tokens to **msg.sender** through **ERC20(assetToken).safeTransfer(msg.sender, _amount)**, but the emitted **TokensWithdrawn** event records the immutable **receiver** as the recipient. The **receiver** is the RWA destination used on the deposit side, not the address that gets tokens back on withdrawal. Off-chain consumers that index **TokensWithdrawn** to track where withdrawn funds went read the wrong destination, which corrupts accounting and monitoring built on this log.

Recommendation

Emit the actual recipient, **msg.sender**, in the **TokensWithdrawn** event.

```
-       emit TokensWithdrawn(block.timestamp, msg.sender, receiver, _amount);
+        emit TokensWithdrawn(block.timestamp, msg.sender, msg.sender, _amount);
```

3S-N04

Discarding the Aave V4 Spoke return values in AaveV4Farm obscures the actual deposited and withdrawn amounts

Id	3S--N04
Classification	None
Category	Suggestion
Status	Addressed in #660e0b1.

Description

AaveV4Farm::depositToAave and **AaveV4Farm::withdrawFromAave** are the keeper-facing entry points that move the farm's **aaveAssetToken** in and out of the Aave V4 Spoke. **depositToAave** approves the spoke and calls **IAaveV4Spoke::supply**, while **withdrawFromAave** calls **IAaveV4Spoke::withdraw**, in both cases passing the caller-supplied **_amount**.

Both spoke functions return a pair of **(uint256, uint256)** values (the shares moved and the corresponding asset amount), and both farm functions discard them. This matters on the withdrawal path because **IAaveV4Spoke::withdraw** does not revert when **_amount** exceeds the value backed by the farm's available shares; it instead caps the output at **min(_amount, previewRemoveByShares(suppliedShares))** and returns the amount actually withdrawn. A keeper who requests an oversized amount therefore receives a silently smaller withdrawal, with no on-chain signal of the realized figure. There is no loss of funds, since the cap simply limits the withdrawal to what the shares cover, but the caller and any off-chain monitoring cannot distinguish a full withdrawal from a capped one without separately querying balances.

This is an operational and observability concern rather than a security flaw.

Recommendation

Have **AaveV4Farm::withdrawFromAave** and **AaveV4Farm::depositToAave** capture and return the **(shares, amount)** pair produced by **IAaveV4Spoke::withdraw** and **IAaveV4Spoke::supply** rather than discarding it, so callers and monitoring tooling can observe the realized amounts directly. Emitting an event with the realized figures, or surfacing them through the return values, also makes a capped withdrawal visible without an additional balance query.

- **function depositToAave(uint256 _amount)**

```
+    function depositToAave(uint256 _amount)
         external
         whenNotPaused
         checkSlippage
         onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
+        returns (uint256 suppliedShares, uint256 suppliedAmount)
     {
         IERC20(aaveAssetToken).forceApprove(spoke, _amount);
-        IAaveV4Spoke(spoke).supply(reserveId, _amount, address(this));
+        (suppliedShares, suppliedAmount) = IAaveV4Spoke(spoke).supply(reserveId,
_amount, address(this));
     }
-    function withdrawFromAave(uint256 _amount)
+    function withdrawFromAave(uint256 _amount)
         external
         whenNotPaused
         checkSlippage
         onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
+        returns (uint256 withdrawnShares, uint256 withdrawnAmount)
     {
-        IAaveV4Spoke(spoke).withdraw(reserveId, _amount, address(this));
+        (withdrawnShares, withdrawnAmount) =
IAaveV4Spoke(spoke).withdraw(reserveId, _amount, address(this));
     }
```

3S-N05

Router allowance is not reset after a partial fill in SwapFarmV2 leaving a standing approval

Id	3S--N05
Classification	None
Category	Suggestion
Status	Addressed in #634368e.

Description

SwapFarmV2::swapWithAggregator lets an account holding the **FARM_SWAP_CALLER** role route the farm's tokens through a whitelisted aggregator router. The actual interaction happens in **SwapFarmV2::_doSwap**, which grants the router an allowance for the full input amount via **IERC20(_tokenIn).forceApprove(_router, _amountIn)**, performs a low-level **call** with the supplied calldata, and then measures **tokensReceived** from the change in **_tokenOut** balance.

The approval is never cleared after the call returns. When the router consumes the entire **_amountIn**, the allowance naturally returns to zero, but if the swap is only partially filled, the router pulls less than **_amountIn** and the difference remains as a standing allowance from the farm to the router.

This is not currently exploitable. The only whitelisted router is KyberSwap, which tolerates residual allowances and, more importantly, enforces **from == msg.sender** on every pull it performs, so a third party cannot use the leftover approval to pull the farm's funds. The concern is forward-looking: if governance later enables additional routers through **SwapFarmV2::setEnabledRouter** whose pull semantics are weaker, or that can be induced to move tokens on behalf of an arbitrary **from**, the standing allowance becomes an avenue for unintended fund movement.

Recommendation

Reset the router allowance to zero at the end of **SwapFarmV2::_doSwap** so that no approval persists beyond the scope of a single swap, regardless of whether the fill was full or partial.

```
IERC20(_tokenIn).forceApprove(_router, _amountIn);
(bool success, bytes memory returnData) = _router.call(_calldata);
require(success, SwapFailed(returnData));
```

```
+       IERC20(_tokenIn).forceApprove(_router, 0);
        tokensReceived = IERC20(_tokenOut).balanceOf(address(this)) -
tokenOutBalanceBefore;
```

3S-N06

Disabling global slippage with a zero value allows swaps on unconfigured token pairs in SwapFarmV2

Id	3S--N06
Classification	None
Category	Suggestion
Status	Addressed in #23822ab.

Description

Farm exposes a single **maxSlippage** parameter that every derived farm uses as the floor for any slippage protection it enforces. **Farm::setMaxSlippage** permits the value to be set to **0**, and the surrounding documentation states that **0** disables slippage checks. In **Farm** itself this behaves as intended, since **mulWadDown(0)** produces a **minAssetsOut** of **0** and the bound is effectively removed.

SwapFarmV2 layers per-pair configuration on top of this global value. The **SwapFarmV2::withValidConfig** modifier guards swaps by requiring that the pair's configured slippage is at least as strict as the global floor: **require(config.maxSlippage >= maxSlippage, InvalidSlippage(...))**. For a token pair that has never been registered through **SwapFarmV2::setSwapPairConfig**, the **PairConfig** struct read from storage is zero-initialized, so **config.maxSlippage** is **0**. When the global **maxSlippage** is also **0**, the check resolves to **0 >= 0** and passes, meaning the modifier no longer distinguishes a configured pair from an unconfigured one. A pair that governance never approved becomes swappable, and because its **cooldown** is likewise **0**, the cooldown branch of the same modifier imposes no restriction either.

The practical reach of this issue is limited. The affected tokens must still be enabled through **Farm::enableAssets** before they can participate, and setting the global slippage to **0** is realistically confined to test environments rather than production configuration. The combination needed to reach the unsafe state is therefore unlikely to occur in normal operation.

Recommendation

SwapFarmV2::withValidConfig can be hardened to reject pairs that were never configured, for example by tracking an explicit **configured** boolean on **PairConfig** and requiring it to be set.